CSE 344 SYSTEM PROGRAMMING HOMEWORK – 1

Name: Muhammed

Surname: Baslak

Student ID: 230104004017

```c
int function_for_nftw(const char *pathname, const struct stat *buf, int type, struct FTW *ftwb); /*The function which is parameter of nftw function.*/
int file_name_comparison(char *file_name);
void get_command(int argc, char *argv[], char *target_path);
void fill_file_info(file_info *fl, const  char *pathname, const struct stat *buf, int type, struct FTW *ftwb);
void convert_permisions_to_string(char *string, mode_t type);
char command_extract_for_alpha(char *command);
int command_extract_for_digit(char *command);
char checking_of_file_type(mode_t type);
void initialized_file_info();
void print();
int file_comparison(file_info *fl);
```

In this homework, some issues were handled: parsing the commant line, file names comparison, using special data type to hold data of given file and other files for comparison, handling command line errors, and so on.

In order to overcome these problems, the course book was mainly used. Also some sources were benefited, which will be listed the last section.

Parsing the Command Line

In order to solve this problem, getopt() function was used. The information about this function was obtained from the book.

```c
#include <unistd.h>

extern int optind, opterr, optopt;
extern char *optarg;

int getopt(int argc, char *const argv[], const char *optstring);
```

See main text for description of return value

Converting File Permition to String Format

For this problem, The example code in the book was benefited:

```
files/file_pe
#include <sys/stat.h>
#include <stdio.h>
#include "file_perms.h"                 /* Interface for this implementation */

#define STR_SIZE sizeof("rwxrwxrwx")

char *          /* Return ls(1)-style string for file permissions mask */
filePermStr(mode_t perm, int flags)
{
    static char str[STR_SIZE];

    snprintf(str, STR_SIZE, "%c%c%c%c%c%c%c%c%c",
        (perm & S_IRUSR) ? 'r' : '-', (perm & S_IWUSR) ? 'w' : '-',
        (perm & S_IXUSR) ?
            (((perm & S_ISUID) && (flags & FP_SPECIAL)) ? 's' : 'x') :
            (((perm & S_ISUID) && (flags & FP_SPECIAL)) ? 'S' : '-'),
        (perm & S_IRGRP) ? 'r' : '-', (perm & S_IWGRP) ? 'w' : '-',
        (perm & S_IXGRP) ?
            (((perm & S_ISGID) && (flags & FP_SPECIAL)) ? 's' : 'x') :
            (((perm & S_ISGID) && (flags & FP_SPECIAL)) ? 'S' : '-'),
        (perm & S_IROTH) ? 'r' : '-', (perm & S_IWOTH) ? 'w' : '-',
        (perm & S_IXOTH) ?
            (((perm & S_ISVTX) && (flags & FP_SPECIAL)) ? 't' : 'x') :
            (((perm & S_ISVTX) && (flags & FP_SPECIAL)) ? 'T' : '-'));

    return str;
}
files/file_pe
```

The code in the book

```
void convert_permisions_to_string(char *string, mode_t type) {

    if(type & S_IRUSR) strcat(string, "r");
    else strcat(string, "-");

    if(type & S_IWUSR) strcat(string, "w");
    else strcat(string, "-");

    if(type & S_IXUSR) strcat(string, "x");
    else strcat(string, "-");

    if(type & S_IRGRP) strcat(string, "r");
    else strcat(string, "-");

    if(type & S_IWGRP) strcat(string, "w");
    else strcat(string, "-");

    if(type & S_IXGRP) strcat(string, "x");
    else strcat(string, "-");

    if(type & S_IROTH) strcat(string, "r");
    else strcat(string, "-");

    if(type & S_IWOTH) strcat(string, "w");
    else strcat(string, "-");

    if(type & S_IXOTH) strcat(string, "x");
    else strcat(string, "-");

}
```

The code in my program

Identify the Type of the File

The required knowledge was obtained from the course slide and the book.

Constant	Test macro	File type
S_IFREG	S_ISREG()	Regular file
S_IFDIR	S_ISDIR()	Directory
S_IFCHR	S_ISCHR()	Character device
S_IFBLK	S_ISBLK()	Block device
S_IFIFO	S_ISFIFO()	FIFO or pipe
S_IFSOCK	S_ISSOCK()	Socket
S_IFLNK	S_ISLNK()	Symbolic link

```
char checking_of_file_type(mode_t type) {

    char r;

    if(S_ISREG(type)) r = 'r';
    else if(S_ISDIR(type)) r = 'd';
    else if(S_ISCHR(type)) r = 'c';
    else if(S_ISBLK(type)) r = 'b';
    else if(S_ISFIFO(type)) r = 'p';
    else if(S_ISSOCK(type)) r = 's';
    else if(S_ISLNK(type)) r = 'l';

    return r;

}
```

Hiding File Information Appropriately for File Comparison

For this issue, a special struct data type was created.

```
typedef struct{

    char file_name[20];
    long int file_size;
    char file_type;
    char file_permision[10];
    int file_number_of_links;

} file_info;
```

Recursively Search Algorithm

In order to solve this problem, nftw () function was used. This funtion get an user-defined function, and according to criteria determined by flags, it applies this function recursively.

```
#define _XOPEN_SOURCE 500
#include <ftw.h>

int nftw(const char *dirpath,
         int (*func) (const char *pathname, const struct stat *statbuf,
                      int typeflag, struct FTW *ftwbuf),
         int nopenfd, int flags);

                  Returns 0 after successful walk of entire tree, or –1 on error,
                        or the first nonzero value returned by a call to func
```

In the created special function, a variable in file_info data type (created special type) is allocated in heap, this variable is filled via fill_file_info() function, and the obtained file data is compared to the file in given command line.

File Name Comparison

The two file names are compared character by character. If '+' sign is encountered in given file name, the index of this string is hold and increasing the index of other file name is continued up to different character from back character from '+' sign.

Other Problems

The functions were generally tested firstly and then they are combined to whole program, but after everything was done, the big error was occured:

```
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

To handle this error, file_info variable was created in heap. Therefore, the stack was relived. In order to learn what this error meant, AI was benefited.

RESOURCES:

KERRISK, Michael, The Linux Programming Interface, 2010

https://www.ibm.com/docs/en/i/7.4.0?topic=ssw_ibm_i_74/apis/stat.htm

https://www.geeksforgeeks.org/c/getopt-function-in-c-to-parse-command-line-arguments/

https://www.tutorialspoint.com/article/how-can-i-get-the-list-of-files-in-a-directory-using-c-or-cplusplus

https://www.ibm.com/docs/en/zos/3.1.0?topic=functions-ftw-ftw64-traverse-file-tree

https://www.youtube.com/watch?v=Pe6sMipTx5k

https://www.youtube.com/watch?v=_r7i5X0rXJk

To learn stack smashing error, the information was taken from AI.